

CASPO: Learned Prefix Value Modeling for Stepwise Reasoning Optimization

TODO: Authors

Draft: April 25, 2026

Abstract

This draft describes CASPO, a reinforcement learning approach for mathematical and structured reasoning tasks that replaces VinePPO-style Monte Carlo rollouts between intermediate reasoning steps with a learned prefix, reward, and value model. The goal is to reduce rollout cost while retaining step-level credit assignment for long-chain reasoning. The method is inspired by the IPVRM formulation of process-aware value/reward modeling, but the exact implementation details, model sizes, datasets, and empirical results remain TODOs. This manuscript is a starting point and intentionally avoids claiming measured gains until experiments are complete.

1 Introduction

Large language models increasingly solve mathematical and programmatic tasks by producing long chains of intermediate reasoning. Reinforcement learning can improve these behaviors, but sparse final-answer rewards make credit assignment difficult. Stepwise reinforcement learning methods such as VinePPO address this by sampling continuations from intermediate prefixes and using rollout-derived estimates to train policies. This can improve the learning signal, but repeated Monte Carlo rollouts between reasoning steps are expensive.

CASPO is intended as a lower-cost alternative. Rather than repeatedly expanding many suffixes from each reasoning prefix, CASPO learns a prefix-conditioned model that predicts reward and value signals for partially completed reasoning traces. The working hypothesis is that learned prefix evaluation can approximate the information supplied by VinePPO-style rollouts while reducing inference overhead during training.

This paper draft makes three contributions to be validated:

1. a formulation of CASPO as PPO-style policy optimization with learned prefix reward/value estimates;
2. an engineering stack for training and inference that separates policy generation, prefix scoring, and rollout orchestration;
3. an evaluation plan for measuring accuracy, sample efficiency, wall-clock cost, and stability on mathematical reasoning benchmarks.

2 Background and Prior Work

PPO. Proximal Policy Optimization (PPO) optimizes a clipped surrogate objective that limits destructive policy updates while using advantage estimates from sampled trajectories [5]. In

language-model reinforcement learning, the action space is token generation and trajectories are generated responses, often regularized against a reference policy. CASPO retains the PPO-style update structure but changes how intermediate reasoning prefixes are valued.

VinePPO. VinePPO proposes using intermediate reasoning states as branch points for additional Monte Carlo continuations [2]. These rollouts can estimate whether a partial chain of thought is likely to lead to a correct final answer. The main practical concern for this project is cost: each prefix may require multiple suffix generations and reward evaluations, which can dominate training time for long responses.

IPVRM. The IPVRM paper motivates learning process-aware reward and value models over intermediate prefixes [3]. CASPO is inspired by this direction: it treats prefixes as first-class objects to score, rather than relying entirely on terminal rewards or repeated online suffix rollouts. The exact mapping from IPVRM terminology to this codebase should be verified against the paper and implementation notes before submission.

GRPO and math RL. Group Relative Policy Optimization (GRPO) and related DeepSeekMath-style training recipes use grouped samples and relative rewards to train mathematical reasoning models without a separate learned critic in some configurations [1]. CASPO can be presented alongside GRPO as another way to improve signal quality for math RL, but its central distinction is prefix-level learned value/reward estimation. This paragraph should be revised once the intended baseline set is finalized.

Inference engines. High-throughput serving systems such as vLLM support efficient batched generation and paged attention [4]. If CASPO uses vLLM or a similar backend for policy sampling, the final paper should describe batching, prefix caching, and scheduling details that affect throughput. TODO: confirm the actual inference backend and version used in experiments.

3 Method

3.1 Problem Setup

Let x denote a problem prompt and $y = (y_1, \dots, y_T)$ denote a generated reasoning trace and answer. A policy $\pi_\theta(y_t \mid x, y_{<t})$ generates tokens autoregressively. A verifier or task reward $R(x, y)$ evaluates the final answer, often producing a sparse scalar reward. For stepwise reasoning, we also consider a sequence of prefixes $p_k = (x, y_{\leq t_k})$ at selected reasoning boundaries, such as sentence, line, or delimiter positions.

3.2 VinePPO-Style Rollout Estimation

In a VinePPO-style estimator, a prefix p_k is expanded with M sampled continuations $\tilde{y}^{(1)}, \dots, \tilde{y}^{(M)}$. The prefix value can be estimated as

$$\hat{V}_{\text{MC}}(p_k) = \frac{1}{M} \sum_{m=1}^M R(x, p_k \oplus \tilde{y}^{(m)}). \quad (1)$$

This estimate can provide useful credit assignment, but the cost scales with the number of prefixes, continuations, and reward evaluations.

3.3 CASPO Prefix Modeling

CASPO replaces most or all of these online Monte Carlo suffix rollouts with a learned prefix evaluator. Given a prefix p_k , the evaluator predicts:

$$\hat{r}_\phi(p_k) \approx \text{local or process reward}, \quad (2)$$

$$\hat{V}_\phi(p_k) \approx \mathbb{E}_{y_{>t_k} \sim \pi_\theta} [R(x, y) \mid p_k]. \quad (3)$$

The evaluator may be trained from terminal rewards, offline rollout labels, human or verifier process annotations, or bootstrapped targets. TODO: specify which targets CASPO currently uses and whether the prefix model shares weights with the policy.

3.4 Policy Objective

For generated trajectories, CASPO computes advantages using a combination of terminal rewards and learned prefix values. A representative PPO objective is:

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (4)$$

where $\rho_t(\theta) = \pi_\theta(y_t \mid x, y_{<t}) / \pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})$. In practice, the loss should also include KL regularization to a reference model, entropy or length controls if used, and value/reward model losses. TODO: match this objective to the exact implementation.

3.5 Training Loop

A conservative training loop is:

1. sample prompts from the training distribution;
2. generate policy responses and identify reasoning prefixes;
3. score final answers with the task verifier;
4. score prefixes with the learned prefix reward/value model;
5. compute token- or segment-level advantages;
6. update the policy with PPO-style clipped optimization;
7. update the prefix model using verified terminal outcomes and any available rollout or process labels.

The final manuscript should clarify whether prefix labels are produced online, cached from a warm-start stage, or periodically refreshed.

4 Training and Inference Stack

CASPO should be described as a system with separable components:

- **Policy model:** TODO: exact base model, tokenizer, context length, precision, and parameter count.

- **Reference model:** TODO: whether KL is computed against the initial policy or a separate frozen model.
- **Prefix evaluator:** TODO: architecture, initialization, scoring granularity, and whether it predicts reward, value, or both.
- **Verifier:** TODO: exact-answer checker, symbolic checker, unit-test harness, or learned judge.
- **Generation backend:** TODO: confirm whether vLLM, Hugging Face generation, or another runtime is used.
- **Distributed training:** TODO: data parallelism, tensor/pipeline parallelism, rollout workers, and scheduling.

The paper should report both training-token throughput and end-to-end wall-clock cost, because replacing online rollouts is primarily a systems and sample-efficiency claim.

5 Experiments and Evaluation Plan

This draft does not include empirical results. The following plan identifies measurements needed before making claims.

5.1 Benchmarks

Candidate benchmarks include GSM8K, MATH, AIME-style held-out problems, and project-specific math or reasoning tasks. TODO: confirm which datasets are licensed, available, and used in the actual runs.

5.2 Baselines

Recommended baselines are:

- supervised fine-tuning only;
- standard PPO with terminal reward;
- VinePPO-style Monte Carlo prefix rollouts;
- GRPO or another grouped relative reward method, if implemented in the same stack;
- CASPO with learned prefix values and ablations.

5.3 Metrics

Report final-answer accuracy, pass@k where appropriate, verifier reward, response length, invalid-answer rate, KL to reference model, training instability events, wall-clock time, generated tokens, and reward-model calibration. For prefix models, calibration plots and prefix-level error against held-out rollout estimates would be useful.

5.4 Ablations

Useful ablations include removing prefix rewards, removing prefix values, varying prefix boundary definitions, varying the amount of offline rollout data used to train the evaluator, and comparing online versus cached prefix scores.

Table 1: Placeholder experiment table. Do not fill with unverified numbers.

Method	Accuracy	Tokens Generated	Wall-Clock	Notes
SFT	TODO	TODO	TODO	TODO
PPO	TODO	TODO	TODO	terminal reward
VinePPO	TODO	TODO	TODO	MC prefix rollouts
CASPO	TODO	TODO	TODO	learned prefix evaluator

6 Limitations

CASPO depends on the quality and calibration of the learned prefix evaluator. If the evaluator assigns high value to superficially plausible but incorrect reasoning, PPO updates may reinforce flawed trajectories. Distribution shift is also likely: as the policy improves or changes style, prefixes encountered during training may drift away from those used to train the evaluator. Additional concerns include reward hacking, verifier brittleness, length bias, and the cost of generating or refreshing prefix-model supervision.

The method should therefore be evaluated with held-out tasks, adversarial or style-shifted prompts when possible, and direct comparisons to rollout-based estimates. Any final claims should separate algorithmic gains from engineering improvements such as batching, caching, or faster inference backends.

7 Conclusion

CASPO is a proposed stepwise reinforcement learning method for reasoning models that uses learned prefix reward/value estimates in place of expensive VinePPO-style Monte Carlo rollouts between reasoning steps. The intended benefit is cheaper credit assignment for long mathematical reasoning traces. The core empirical questions are whether learned prefix estimates are accurate enough to guide PPO updates, whether they reduce training cost at equal or better accuracy, and how robust they remain as the policy distribution changes.

References

- [1] TODO: Verify authors. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, 2024. TODO: verify arXiv identifier, author list, and whether this is the right GRPO citation.
- [2] TODO: Verify authors. TODO: Verify title for vineppo. *arXiv preprint arXiv:2410.01679*, 2024. Placeholder metadata. TODO: verify title, authors, version, and venue.
- [3] TODO: Verify authors. TODO: Verify title for ipvm. *arXiv preprint arXiv:2604.13197v1*, 2026. Placeholder metadata. TODO: verify exact title, authors, and version.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. TODO: verify proceedings details and page numbers.

- [5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. TODO: verify final bibliographic metadata before submission.